

Program:

Sequence of instructions given to a computer by a human in order to solve a problem

programming:

Art of making a computer do what you want it to do

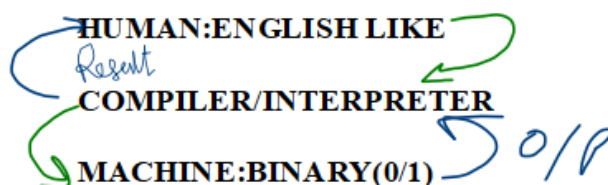
Why do you call it an art?

A human is trying to utilize his or her skill in thinking and utilize computer skill in doing.

A human is more intelligent than a computer but a computer is much faster than a human.

programming language:

Language used and understood by humans (English-like) but also understood and processed by computers.



Generations of programming languages :

1G

Machine language or a binary language

2G

Assembly-level language: opcode operation codes to be fed, more of a hardware language

3G

High-level programming language, English-like language, translated and gave us WORA(Write Once Run Anywhere).

4G

SQL(Structured Query Language, a specialized language for databases, capable of giving CRUD)

5G

Natural language processing (NLP), prompt engineering, and AI.

C++:3g

C++ is a successor of C, which was procedure-oriented. This had optional object-oriented programming. That means in C++ you can write code in classes and objects but can also write normal code. Currently the latest version of C++ is 2021 and it has been upgraded from 1983 till now.

If you are using C++ as a language for an interview coding round, must-know topics are STL and RTL.

C++ was the last language to have a powerful concept called a pointer, after which no language ever had that pointer. They all have references.

Working:

Job of compiler and interpreter :

A compiler and interpreter work as a translator from a high-level English-based language to a low-level machine-understandable language.

It works in multiple passes or in multiple stages.

It tokenizes the data, converts it to needed object code, and later converts it to the needed machine's native code(Machine's own).

It's a compilation-based language where code is first compiled and object code is created. Later on object code is fed and compiled back for running or an EXE file.

Difference between compilation and interpretation

In compilation the entire code is converted and then used. If there are errors a list of errors will be generated before exchange or interchange.

+ve:Fast because the entire code is fed

-ve:A developer has to deal with multiple errors to handle or solve.

In interpretation the conversion happens line by line. Each line is read, converted, and given to the computer as an end result. If there is an error on the first line, the system will stop.

Language:

Alphabets were the first thing.
A-Z

Words .

Sentences, which are collections of words

Paragraphs, essay, etc.

Programming:

character set: ASCII(8bit):Eng
A-Z,a-z,0-9,!@#\$\$%.....

Tokens:

Words made from the character set of C++, smallest unit of words.

3 basic types of statements for understanding programs:

- sequence
- branch
- loops

Program

Token: Words made from C++ character set

Keyword:

Words with reserved(Special meanings) meanings can not be used otherwise.

Common examples of keywords: if, else, while, for, break, etc.

ex: Data Types:

Identifiers:

Variables or identifiers are symbolic identifications given to some memory location where data will be stored.

ex: Home:

Operators:

Datatypes:

Will describe the property of data one can store .
only numbers (Natural numbers +ve and -ve)

byte 1B

short 2B

int 4B

long int 8B

Real numbers with a decimal point

float 4B

double 8B

long double 10B

Any character from the C++ character set (alphabets, symbols, or even digits) can fall in this.

char 'a' 1B

string "amar" n-Bytes

boolean

0(false)/1(true)

Why are there several subcategories available for each type?

That is because each one has a different memory requirement depending on the data.

One could select the right amount of type for the right amount of data.

Signed

$$-2^{\frac{n}{2}} \text{ to } +2^{\frac{n}{2}} - 1$$

Byte

$$-128 \text{ to } +127 (0.127)$$

$\% \Rightarrow$ It returns the remainder of division.

$$5 \% 3 = 2$$

$$\begin{array}{r} 1 \\ 3 \overline{) 5} \\ \underline{-3} \\ 2 \end{array}$$

$$17 \% 2 = 1$$

$$\begin{array}{r} 8 \\ 2 \overline{) 17} \\ \underline{-16} \\ 1 \end{array}$$

$$50 \% 118 = 50$$

$$\begin{array}{r} 0 \\ 118 \overline{) 50} \\ \underline{0} \\ 50 \end{array}$$

$$10 \% 10 = 0$$

$$\begin{array}{r} 1 \\ 10 \overline{) 10} \\ \underline{-10} \\ 0 \end{array}$$

Operators:

Arithmetic

+

-

*

/

%

High: $*/\%$ 1st
Low: $+ -$ 2nd

Rule of precedence.

First if all operators have equal precedence, go from left to right.

$$5 + 2 - 3 + 8 \Rightarrow 12$$

$$7 - 3 + 8 \Rightarrow 4 + 8 = 12$$

Second if different precedents: first saw higher precedence than lower precedence.

$$5 + 2 * 3 - 6 = 5 + 6 - 6 = 11 - 6 = 5$$

Third, in case of brackets start from innermost and go to outermost.

$$((5 + 2) * (3 - 6))$$

$$(7 * -3)$$

$$-21$$